

Chapter → 7

Methods

Sometimes our program grows in size and we want to separate the logic of main method to other method

for instance → if we are calculating average of a numbers pair 5 times, we can use the methods to avoid repeating the logic.

DRY → Don't Repeat Yourself

● Syntax of a method

A method is a function written inside a class. Since Java is an object oriented language, we need to write the method inside same class.

```
dataType name()
```

```
{
```

```
// Method body
```

```
}
```

Following method returns sum of two numbers.

```
int mysum (int a, int b)
```

```
{
```

```
int c = a + b;
```

```
return c;
```

```
}
```


//code

Public class METHODS

{
static int mysum(int x, int y)

{

int z;

if (x > y)

{

z = (x + y) * 5;

}

else

{

z = (x + y);

}

return z;

}

public static void main (String [] args)

{

int a = 5;

int b = 10;

int c;

c = mysum(a, b);

int a1 = 7;

int b1 = 5;

int c1;

c1 = mysum(a1, b1);

System.out.println(c);

System.out.println(c1);

}

}

● Calling a method

1. A method can be called by creating an object of the class in which the method exists followed by the method call.

Calc obj = new Calc (); → object creating
obj.mySum(a,b); → Method call upon an object.

2. The values from the method call (a and b) are copied to the a and b of the function mySum.
3. Thus even if we modify these values a and b inside the method, the values in the main method will not change.

● Void Return Type

4. When we don't want our method to return anything we use void as the return type.

```
public class VOIDTYPE
{
```

```
    static void tellJoke ()
    {
```

```
        System.out.println("I invented a new word!\n" +
                             "Plagiarism!");
    }
```

```
    public static void main (String[] args)
    {
```

```
        tellJoke ();
    }
```


● Static Keyword

Static keyword is used to associate a method of a given class with the class rather than the object.

Static method in a class is shared by all the objects.

NOTE: In case of Arrays, the reference is passed. Same is the case for object passing to methods.

//code

```
public class REFERENCEMETHOD
```

```
{
    static int change(int a)
    {
```

```
        a = 98;
    }
```

```
    static void change2(int [] arr)
    {
```

```
        arr[0] = 98;
    }
```

```
    public static void main(String [] args)
    {
```

```
        // case 1: changing the integer
```

```
        int x = 46;
```

```
        change(x);
```

```
        System.out.println("The Values of x after running change is: " + x);
```

```
        // case 2: changing the array
```

```
        int [] marks = { 52, 73, 77, 83, 95, 100 };
```

```
        change2(marks);
```

```
        System.out.println("The value of x after running change is: " + marks[0]);
    }
```


★ Method Overloading

Two or more methods can have same name but, different parameters. Such methods are called overloaded methods.

```
void foo()
```

```
void foo (int a)
```

```
int foo (int a, int b)
```

⇒ Overloading function
foo

Method overloading cannot be performed by changing the return type of methods.

```
public class METHODOVERLOADING
```

```
{
```

```
    static void foo ()
```

```
    {
```

```
        System.out.println("Good Morning bro!");
```

```
    }
```

```
    static void foo (int a)
```

```
    {
```

```
        System.out.println("Good Morning" + a + " bro!");
```

```
    }
```

```
    static void foo (int a, int b)
```

```
    {
```

```
        System.out.println("Good Morning" + a + " bro!");
```

```
    }
```

```
    public static void main (String [] args)
```

```
    {
```

```
        foo ();
```

```
        foo (3000);
```

```
        foo (3000, 4000);
```

```
    }
```


● Variable Arguments (Varargs)

A Function with Varargs can be created in Java using the following syntax:

```
public static void foo (int... arr)
{
    // arr is available here as int[] arr.
}
```

foo can be called with zero or more arguments.
like this:

```
foo(). foo(7, 8, 9) foo(1, 2, 7, 8, 9)
```

//code

```
public class VARARGS
{
    static int sum (int... arr)
    {
        int result = 0;
        for (int a : arr)
        {
            result += a;
        }
        return result;
    }

    public static void main (String[] args) {
        System.out.println("Welcome to Varargs Tutorial");
        System.out.println("The sum of 4 and 5 is : " + sum(4, 5));
        System.out.println("The sum of 4, 3 and 5 is : " + sum(4, 3, 5));
        System.out.println("The sum of 2, 4, 3 and 5 is : " + sum(2, 4, 3, 5));
    }
}
```


★ Recursion

A function in Java can call itself. Such calling of function by itself is called recursion.

Example: Factorial of a number.

$$\text{Factorial}(n) = n * \text{Factorial}(n-1) \quad \forall n \geq 1$$

Quick quiz:

Write a Java program to calculate (recursion must be used) factorial of a number in Java.

```
public class FACTORIAL
```

```
{
```

```
    static int Factorial(int n)
```

```
    {
```

```
        if (n == 0 || n == 1)
```

```
        {
```

```
            return 1;
```

```
        }
```

```
        else {
```

```
            return n * Factorial(n-1);
```

```
        }
```

```
    }
```

```
    public static void main (String[] args)
```

```
    {
```

```
        int n = 5;
```

```
        System.out.println("The value of factorial is: " +
```

```
        Factorial(n));
```

```
    }
```

```
}
```


Case II:

```

public class FACTORIAL
{
    static int iterative_factorial (int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else {
            int product = 1;
            for (int i = 1; i <= n; i++)
            {
                product *= i;
            }
            return product;
        }
    }

    public static void main (String args[])
    {
        int n = 6;
        System.out.println("The value of factorial is: " +
                           iterative_factorial(n));
    }
}

```